The University of Sydney Business School

QBUS6810: Machine Learning in Business

A data-driven analysis to predict delivery time

Project Group 29:

540615324

430024450

540154801

540321469

530263829

AI Acknowledgement *4*

Executive summary *5*

Problem formation & Business understanding *5*

Business problem definition **5**

Justification for machine learning **5**

Business success metric **6**

Exploratory data analysis (EDA) *6*

Feature engineering *11*

Missing values **11**

Handling outliers **12**

Features created **12**

Feature transformation **12**

Feature Selection Process **13**

Model Development & Algorithm Understanding *14*
Model Evaluation *16*
Evaluation Framework & Appropriate Metrics **16**
Evaluation Results **17**
Business implications **19**
Business insights *20*
Key business insights **20**
Actionable recommendations **20**
Limitations of insights and assumptions **21**
Business implementation & Impact *22*
Implement Plan **22**
Business Impact **22**
Reference *24*
Appendix A: Model *25*
Appendix B: Teamwork & project execution *25*
Group Reflection **26**
a. Biggest Challenge: 26
b. Most Impactful Strategy 27
c. Key Takeaway for Your Future Career 28
Individual Reflection **28**
Appendix C: Additional tables and figures *30*

Table of Contents

AI Acknowledgement *2*
Executive summary *3*
Problem formation & Business understanding *3*
Business problem definition **3**
Justification for machine learning **3**
Business success metric **4**
Exploratory data analysis (EDA) *4*
Feature engineering *9*
Missing values **9**
Handling outliers **10**

Features created **10**

Feature transformation **10**

Feature Selection Process **11**

Model Development & Algorithm Understanding ***12***

Model Evaluation ***14***

Evaluation Framework & Appropriate Metrics **14**

Evaluation Results **15**

Business implications **17**

Business insights ***18***

Key business insights **18**

Actionable recommendations **18**

Limitations of insights and assumptions **19**

Business implementation & Impact ***20***

Implement Plan **20**

Business Impact **20**

Reference ***22***

Appendix A: Model Evaluation ***23***

Appendix B: Teamwork & project execution ***23***

Group Reflection **24**

a. Biggest Challenge: 24

b. Most Impactful Strategy 25

c. Key Takeaway for Your Future Career 26

Individual Reflection **26**

Appendix C: Additional tables and figures ***28***

AI Acknowledgement

This assignment was prepared with the assistance of generative AI tools, specifically ChatGPT (OpenAI, 2025) and Jupyter Notebook Anaconda Assistant. AI support was used to:

- Refine the structure and flow of written sections.
- Debugging, fixing codes, and improving visualisations used in the report

All analytical decisions, interpretations of results, and conclusions presented in this report remain the responsibility of the author. The AI tool functioned as a writing and editing assistant, helping to improve clarity, readability, and consistency, while the critical thinking, data analysis, and final judgment were conducted by the authors.

Executive summary

This project aims to help **Olist**, a Brazilian e-commerce platform, enhance its logistics performance and customer satisfaction by predicting delivery time using machine learning. Through extensive data integration and analysis of over 99,000 orders, the study identifies that distance, freight value, and purchase timing are the primary drivers of delivery duration.

After data cleaning, feature engineering, and model experimentation, three models were benchmarked - **Decision Tree** (baseline), **LightGBM**, and a **Stacked LightGBM + GAM** model. The stacked model achieved the best results (**MAE = 5.13 days**, **RMSE = 8.05**, **R^2 = 0.26**), providing accurate, stable, and interpretable predictions.

SHAP analysis confirms that longer distances, peak-season purchases, and heavy freight significantly increase delivery times. These insights support actionable business strategies such as regional warehousing, carrier diversification, capacity planning during peak periods, and tiered shipping strategies for bulky items.

By integrating the model into Olist's logistics system, the company can improve delivery forecasts, reduce customer complaints, and enhance operational efficiency, directly strengthening customer trust and competitive advantage.

Problem formation & Business understanding

Business problem definition

Olist is a Brazilian company that helps small and medium-sized businesses manage multiple sales channels through integrated ERP, logistics, and point-of-sale (POS) systems. Between 2016 and 2018, Olist supported over 3,100 sellers across 23 states, processing around 100,000 orders. In e-commerce, efficient logistics and accurate delivery estimation are essential for maintaining customer satisfaction and operational performance. Inaccurate or delayed deliveries can lead to poor shopping experience, negative feedback, loss of trust, and eventually reduced competitiveness. As small and medium enterprises increasingly rely on online platforms, strong supply chain management (SCM) practices become vital for sustaining growth (Kherbach & Mocan, 2016).

The central business question is: *Can we accurately predict delivery delays using machine learning to improve logistics efficiency and customer satisfaction?*

Justification for machine learning

For Olist, improving delivery time prediction directly enhances logistics planning, cost control, and customer satisfaction. However, traditional forecasting methods cannot account for the many uncertain factors influencing delivery performance, such as geographical distance, product type, order timing, and carrier variability.

Machine learning (ML) provides a scalable, data-driven approach capable of processing large historical datasets, identifying complex relationships, and adapting to changing conditions. Evidence from Facey (2023) shows that DHL used ML to achieve last-mile delivery route optimization, providing customers with real-time updates and improving first-attempt delivery success rates. Similarly, Olist can use ML to predict delivery delays, optimize logistics, and enhance customer communication.

ML also supports data integrity and proactive risk management by detecting errors, handling missing data, and predicting disruptions (Balan et al., 2025). These insights enable better decision-making and greater supply chain agility.

Nonetheless, several challenges must be considered. Model accuracy depends on data quality, timestamp consistency, and location synchronization recording across systems. Moreover, operational variability factors such as weather, carrier performance, and regional infrastructure may still introduce novel uncertainty. Finally, the model requires continuous retraining to adapt to evolving business conditions and maintain predictive accuracy.

Business success metric

To evaluate the tangible business value generated by implementing the machine learning model in Olist's logistics operations, a set of key success metrics has been defined. These indicators directly align with Olist's strategic objectives of improving efficiency, accuracy, and customer satisfaction (Table 1).

Metrics	Definition	Strategic business goals
NECR (New-to-Existing Customer Ratio)	The proportion of orders coming from new customers relative to returning customers	Reflecting Olist's effectiveness in customers' acquisition and market expansion.
DTR (Delivery Time Reduction)	The average reduction in delivery time compared to historical baseline.	Indicates operational efficiency gains and cost savings.
DDR (Delivery Delay Rate)	The proportion of orders delivered later than the promised time window.	Measures operational efficiency by reducing late deliveries to improve customer satisfaction.
OTDR (On-Time Delivery Rate)	The percentage of orders delivered within the promised time window	Captures the reliability and consistency of logistics operations.
MASE (Mean Absolute Shipping Error)	The average deviation between predicted and actual delivery times	Evaluates the predictive accuracy of the ML model and its contribution to data-driven logistics planning.

Table 1. Business Success Metrics

Exploratory data analysis (EDA)

A comprehensive set of datasets provided by Olist were use in this project, which provided information on nearly **99,441** e-commerce transactions made on Olist website in 04/09/2016-17/10/2018 period. Those datasets include:

- Customers: Contains unique customer identifiers and location details.
- Geolocation: Provides geographical coordinates associated with zip code prefixes.
- Orders: Includes order statuses, timestamps, and related customer and seller IDs.
- Items: Details products within each order, including pricing and shipping information.
- Order Payments: Records payment methods and transaction amounts.
- Order Reviews: Captures customer reviews, ratings, and feedback timestamps.
- Products: Lists product specifics such as category, dimensions, and weight.
- Sellers: Contains seller identifiers and location details.
- Product Category Translation: Translates product category names from Portuguese to English

<u>Key findings:</u>

1. **<u>Orders dataset</u>**

The Orders dataset shows that more than **97%** of orders were successfully delivered from 11/10/2016 to 17/10/2018, while the remaining 3% were either still in progress or never completed (missing values in *"order_delivered_customer_date" column)*. Among the delivered orders, **91.88%** arrived on or before the expected date, whereas **8.12%** experienced delays. Since the goal of this research is to predict delivery time, the 3% uncompleted orders will be excluded from the dataset when training the model.

The overall **delivery time** (measured from when a customer places an order until it is received) ranges widely from less than a day to approximately 210 days, though most deliveries occur within around 10 days (Figure 1). This represents the total waiting time from the customer's perspective throughout the entire e-commerce process.

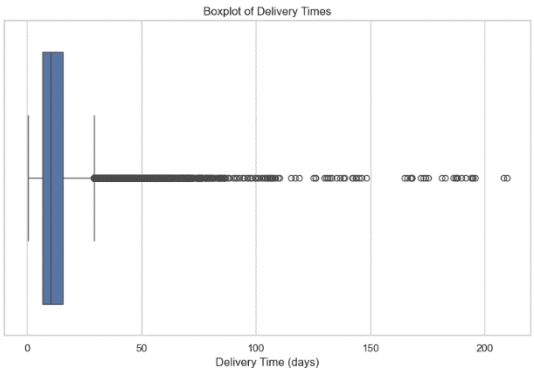


Figure 1. Distribution of total delivery time

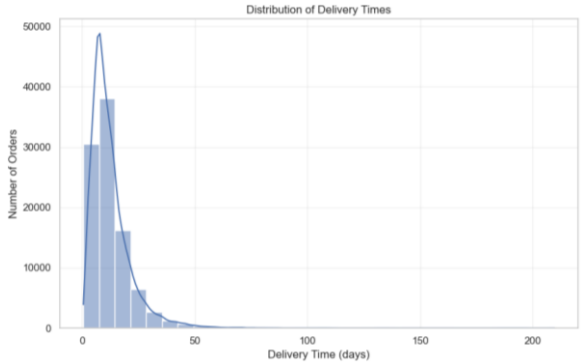


Figure 2. Distribution of Processing and Shipping Duration

When decomposed into processing time (from order placement to carrier pickup) and shipping time (from carrier pickup to delivery), it is observed that processing is generally fast, typically completed within a few days (generally 2 days). In contrast, shipping shows greater variation and tends to take longer (generally a week), suggesting that the main bottleneck lies in the shipping stage. Nevertheless, a small number of extreme outliers exist, where either stage can take hundreds of days to complete (Figure 2).

A small number of records show negative processing or shipping durations **(165 rows** and **23 rows,** respectively**)**, resulting from timestamp inconsistencies in the source data. These issues are likely caused by mis-ordered timestamps, time zone discrepancies, or system synchronization delays between different data sources (e.g., order system and carrier API). Such inconsistencies lead to cases where the carrier receipt appears to occur before the purchase time, or the delivery happens before the carrier receive the package. These entries were excluded from the duration analysis to ensure accuracy but retained in the model training dataset to preserve overall data completeness.

While it is crucial to provide customers with an accurate estimated delivery date, the difference between the estimated and the actual delivery date also experiences the same variation. Figure 3 shows the variation in the differences between estimated and actual delivery time. Orders can be delivered to the customer as early as 146 days in advance and can be delayed as many as 189 days later.

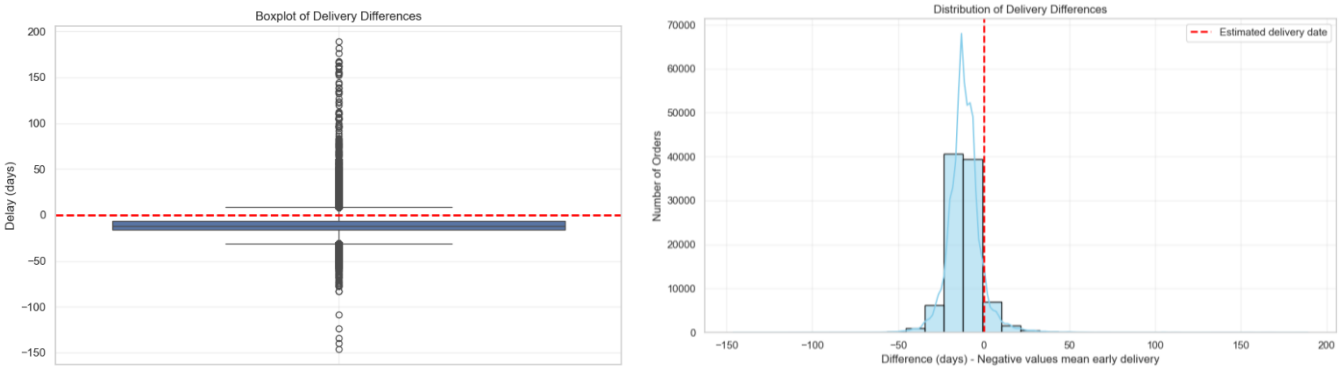


Figure 3. Distribution of Differences in Estimated and Actual Delivery Time

2. <u>Payments dataset</u>

In the Payments dataset, **103886** individual payments were associated with **99440** unique order ids. The most popular payment methods are **credit card** (approximately 74%), followed by *boleto* with 19%. Boleto is a famous "ticket"-like payment method in Brazil, where it acts like a cash-alternative method using a printed or virtual barcode as a cash voucher (Stiltner, 2024). After aggregating multiple payment records at the order level, the mode was used to determine the primary payment method for each order, as some orders involve multiple payment types. Regarding the relationship between payment method and delivery time, only debit card payments show a relatively shorter delivery duration, while other payment methods exhibit no clear difference in average delivery time. Moreover, there is a moderate positive relationship between freight value/total price per order and delivery time (see Appendix).

Most buyers paid for their orders all at once, some needed 2-5 instalments to finish. However, the number of instalments varies up to 29 payments. This is likely due to higher order amount. This is further proven by the right-skewed distribution of the total payment value for each order. Transaction values range from 0 up to 13664 in Brazilian Real, as shown in Figure 4.

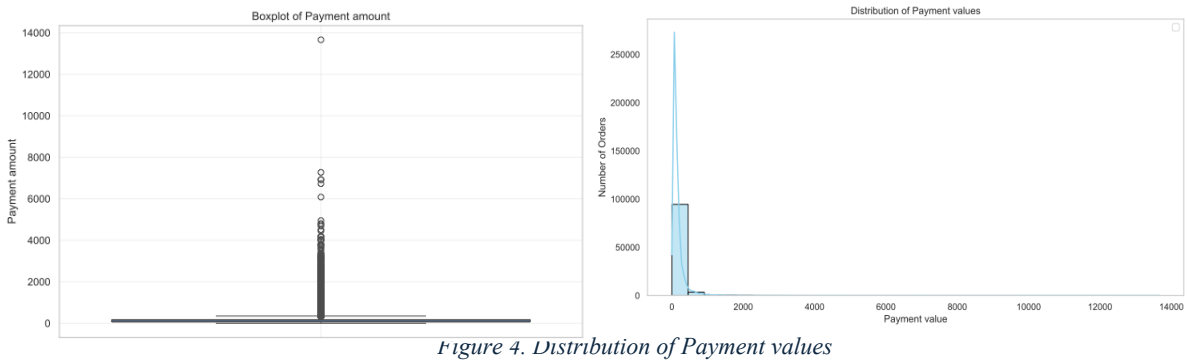


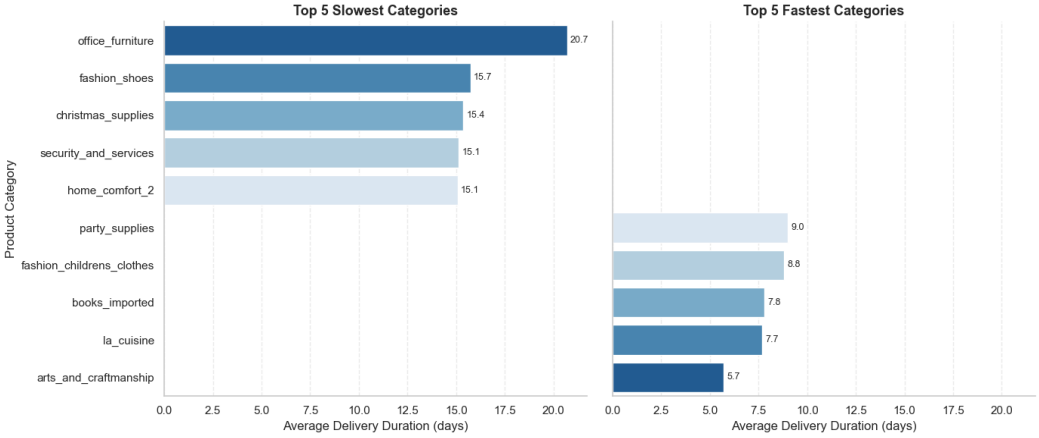
Figure 4. Distribution of Payment values

3. Order_items dataset

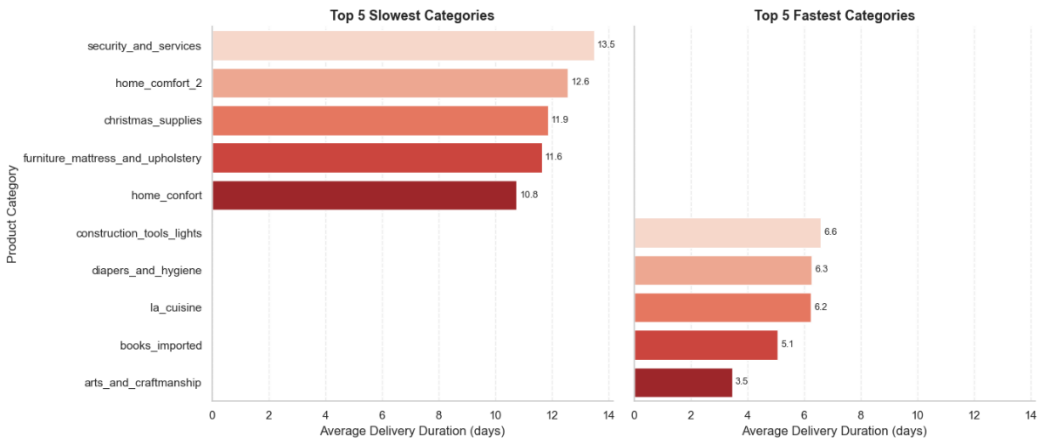
Based on the examination of the Items dataset, each order ID can correspond to more than one product from more than one seller. There are 98,666 unique order IDs, which is fewer than the total number of order IDs in the Orders dataset. This discrepancy suggests a potential data quality issue related to information loss during dataset merging or fragmentation. Additionally, several features, such as the number of items per order, price, and freight value, exhibit strong **right-skewed** distributions, indicating that most orders involve low values while a small number of orders account for exceptionally high ones. This indicates the need to carefully handle outliers and transform the variable before modeling, such as applying log-transformation, scaling, or standardizing.

4. Products dataset

There are **72** distinct product categories in the dataset, and delivery speed varies notably across them. For total delivery duration (Figure 5)**,** categories such as *office_furniture* (20.7 days), *fashion_shoes*, and *christmas_supplies* exhibit the longest delivery times, while *arts_and_craftsmanship*, *la_cuisine*, and *books_imported* are among the fastest (averaging 6–9 days). Larger or more fragile items, like furniture and special-purpose goods, generally take longer to prepare and ship, while smaller items are fulfilled more efficiently. It is also worth noting that seasonal goods, such as *christmas_supplies*, may have longer delivery times because they are often pre-ordered well before the occasion or may experience shipping delays due to high traffic during peak seasons**.** In contrast, products with a perishable nature, such as *cuisine*, are typically prioritized for faster delivery to maintain freshness and customer satisfaction.



When focusing specifically on **shipping duration** (Figure 6)**,** *security_and_services*, *home_comfort_2*, and *christmas_supplies* remain the **slowest categories**, taking around **12–14 days** on average. In contrast, *arts_and_craftsmanship*, *la_cuisine, books_imported*, and *diapers_and_hygiene* are delivered the fastest, typically within **3–6 days.** These products are typically small, lightweight, and easier to package and transport, which contributes to their shorter delivery times. This pattern reinforces that categories mainly consist of bulky or heavy goods require special handling or longer logistics chains, explaining their extended delivery times**.** Product size, weight, and handling complexity can be key players in predicting delivery time.



5. Customer Seller and Geolocation datasets

Total delivery times vary substantially across both **customer** and **seller** states, due to the fluctuation of the seller's processing duration and the shipping duration. In **Figure X**, which shows *average processing duration by seller state*, the majority of states have processing times between **2 and 4 days**, indicating relatively consistent seller efficiency nationwide. However, states such as *Maranhão (MA)* and *Mato Grosso do Sul (MS)* show slightly longer averages (around **4–5 days**), suggesting possible delays in order preparation or carrier pickup processes.

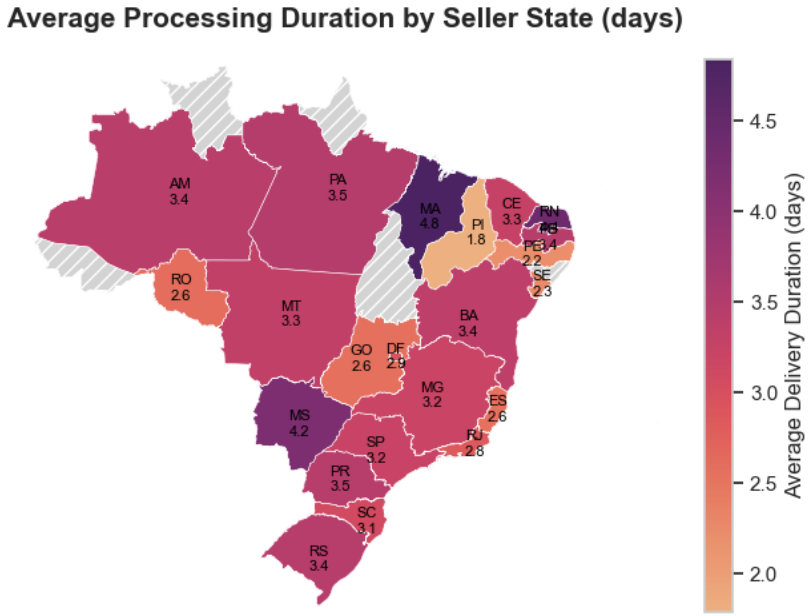
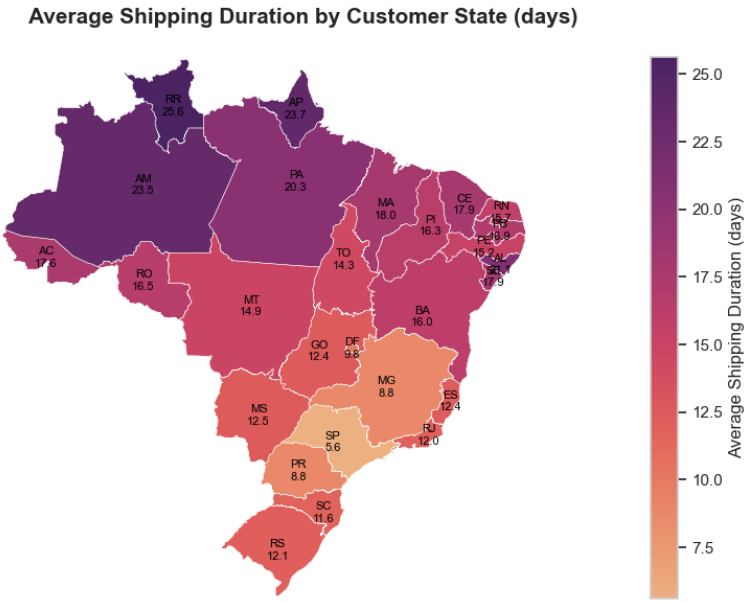


Figure 5. Average Processing Duration by Seller State



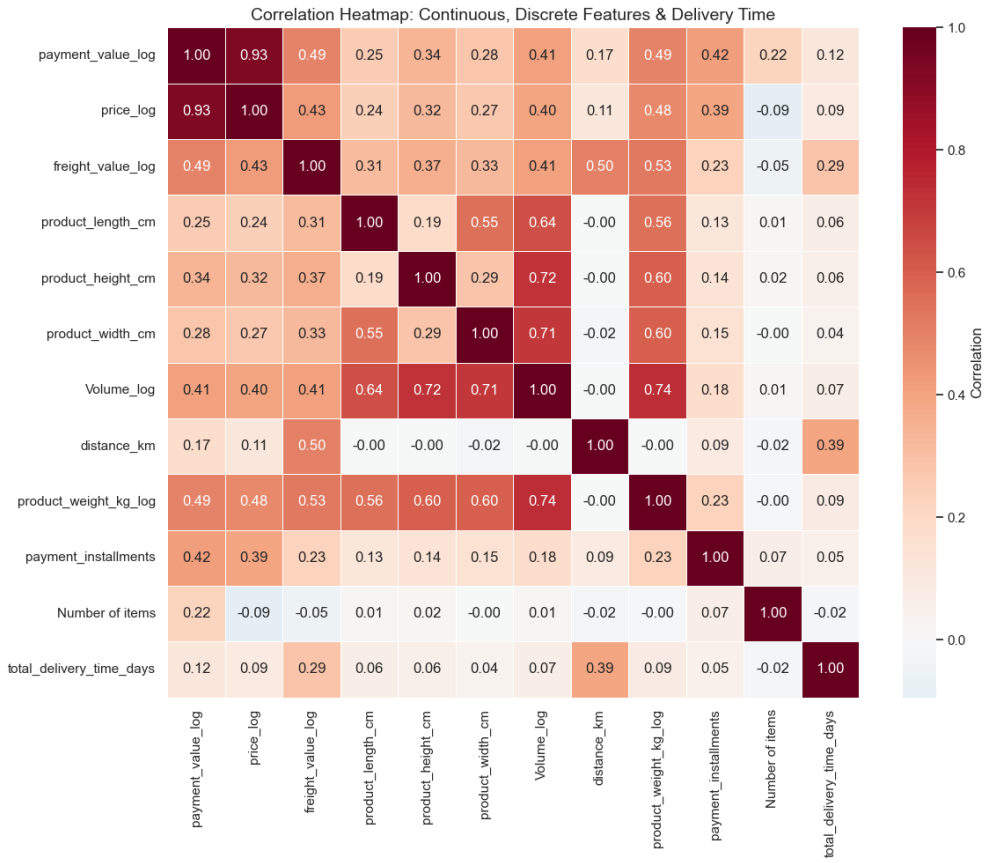
In contrast, **Figure Y** reveals much larger regional differences in *average shipping duration by customer state*. Customers in **northern and remote states**, including *Roraima (RR)*, *Amapá (AP)*, and *Amazonas (AM)*, experience significantly longer shipping times (over **20 days**) compared to customers in **south-eastern states** such as *São Paulo (SP)*, *Paraná (PR)*, and *Minas Gerais (MG)*, where orders arrive within **6–9 days**. This highlights that **logistical distance and infrastructure quality** play a much greater role in delivery time variability than seller-side processing efficiency.

6. Correlation between predictors and target

After merging relevant information from fragmented datasets, the set of predictors was analysed in relation to total delivery time. The correlation heatmap (Figure X) illustrates relationships among continuous and discrete features with total delivery time. Payment value, price, and freight value show strong positive correlations with each other (above 0.9) and with product' dimensions, reflecting their direct monetary relationship. Product dimensions, particularly volume and weight, are also highly correlated (0.6–0.7), suggesting that larger items are typically heavier and cost more to ship. This finding aligns with earlier insights that furniture and bulky items generally take longer to deliver due to higher cost and logistical complexity.

Among all predictors, distance between the costumers and the sellers **(0.39)** shows the strongest correlation with delivery time, indicating that longer shipping distances modestly

increase delivery duration. In contrast, other features such as product dimensions, number of items, and payment instalments display only weak correlations (below 0.1), implying that these factors have minimal direct influence on delivery time compared to distance and freight-related variables. These correlations will serve as the foundation for feature selection in the subsequent model-building process, helping to identify the most relevant predictors that influence delivery performance.



Feature engineering

Missing values

Missing values were removed, imputed, or retained depending on their context and relevance to model development. Records with missing delivery-related timestamps (roughly 3% of the dataset), such as *order_approved_at*, *order_delivered_carrier_date*, and *order_delivered_customer_date*, were excluded from the training dataset, as they typically represent incomplete orders (either cancelled or still in progress).

Approximately **1.2%** of product items lacked dimension attributes, which were imputed using the median to preserve data integrity while minimizing the influence of outliers. For the *product_category_name* variable, after merging with its English translation, **610** missing entries remained and were replaced with the label "Unknown."

Following dataset integration, a small number of additional missing values (fewer than 500 entries) were detected due to mismatched information across sources. These were imputed

using the median for continuous variables (to account for skewed distributions and outliers) and the mode for categorical features, ensuring a consistent and complete dataset for subsequent modelling.

Handling outliers

Extreme values were identified in several variables, including *payment_value, price, freight_value,* and *product dimensions*. Given the wide range of product categories, these outliers likely represent legitimate variations associated with certain product types, such as furniture or bulky items, rather than data errors. Therefore, the outliers were retained in the training dataset to preserve information diversity. However, a logarithmic transformation was applied to these variables to reduce skewness and stabilize variance, ensuring more balanced feature distributions for model training.

Features created

The following features were created or derived to support exploratory and predictive analysis:

- Delivery_time_days – Calculated as *delivery_date − purchase_timestamp* to represent the total time taken to deliver an order to the customer. This variable applies only to orders with a *"delivered"* status and serves as the target variable for modeling.
- Processing_duration (days) and Shipping_duration (days) – Represent the seller's processing time and the carrier's actual shipping time, computed as *carrier_delivered − purchase_timestamp* and *customer_delivered − carrier_delivered*, respectively. These were used for EDA but excluded from the predictive model.
- Delivery_difference_days – The difference between the estimated and actual delivery dates. Negative values indicate early delivery, zero indicates on-time delivery, and positive values indicate delays. This variable was used for analysis but not included in modeling.
- Product_volume (cm^3) – Calculated as *height × width × length* to estimate package size.
- Number_of_items – Aggregated count of items per *order_id* across the dataset.
- Carrier_weekday and Carrier_month – Categorical variables extracted from the timestamp indicating when the carrier received the package.
- Purchase_weekday and Purchase_month – Categorical variables extracted from the timestamp of when the customer placed the order.
- Delivery_is_weekend and Purchase_is_weekend – Binary indicators showing whether delivery or purchase occurred on a weekend.
- Distance_km: The geodesic distance between the seller and the customer, calculated from their geolocation coordinates.

Feature transformation

Order-level aggregation:
Since the target variable represents *delivery time per order*, data from related tables (e.g., payments, products, and *order_items*) were aggregated by *order_id* to align features with delivery duration. Product weight and volume were summed or averaged per order, while price, freight value, and payment details were also aggregated to ensure each order is represented by a single record.

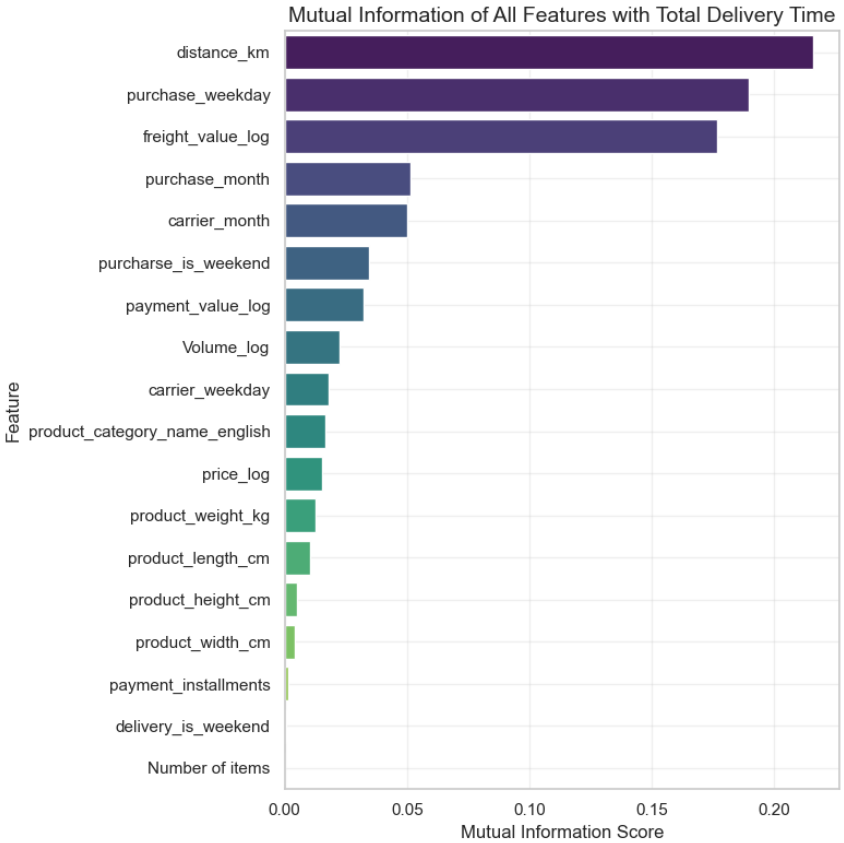
Unit conversion:
Product weight was converted from grams to kilograms for consistency and easier interpretation, without affecting model performance.

Category encoding:
Label encoding was applied to *product_category_name_english* to convert categories into numeric form, suitable for tree-based models and avoiding the high dimensionality of one-hot encoding.

Log transformation:
A log transformation was applied to *payment_value*, *freight_value*, *price*, *product_volume*, *product_weight_kg*, and *distance_km* to reduce right-skewness and lessen the impact of extreme outliers, thereby improving model stability and interpretability for linear regression models. This transformation was not applied to tree-based models, which are inherently more robust to skewed feature distributions.

Temporal features:
Month, weekday, and weekend indicators were extracted from purchase and carrier timestamps to capture both customer behavior and logistical timing effects, as weekend or peak-period orders may face longer processing times.

Feature Selection Process



The mutual information analysis (Figure X) identifies which features share the strongest relationship with total delivery time. The most influential predictors are *distance_km, purchase_weekday*, *and freight_value_log*, indicating that delivery duration is primarily driven by geographical distance, order timing, and shipping cost factors. Features such as *purchase_month, carrier_month*, and whether the purchase occurred on a weekend also contribute moderately, suggesting seasonal and operational influences on delivery speed.

In contrast, product-level features (e.g., *dimensions, weight*, and *category*) and transactional attributes (e.g., *number of items, installments*) exhibit low mutual information scores, implying that their direct impact on delivery time is minimal compared to logistical and temporal factors.

Model Development & Algorithm Understanding

Data-driven feature discovery and preprocessing

To reduce dimensionality and fitting the model with business logic before intensive training, we made two complementary relevance screens on the full dataset (>90,000 orders), all based on the feature engineering output. First, a Pearson correlation matrix against *total_delivery_time_days* identified linear signal, features with $|r| \geq 0.10$ were flagged as candidates for inclusion. Second, we computed mutual information (MI) to capture non-linear associations and retained variables with MI ≥ 0.015 (Cover & Thomas, 2006). Applying these thresholds yielded seven predictors with consistent operational interpretation: distance_log, freight_log, payment_value_log, purchase_weekday, purchase_month, purchase_is_weekend, and Volume_log.

Positively supported, right-skewed quantities (distance_km, freight_value, payment_value, and the geometric volume) were log-transformed to stabilize variance, reduce the influence of extreme values, and improve both tree partitioning and spline smoothness (Hastie, Tibshirani, & Friedman, 2009). All downstream models consumed these engineered features only.

Experimental protocol and evaluation design

To iterate quickly without compromising generalization assessment, we adopted a subsample + strict holdout procedure. For training each model we drew 20% of the data, then split it 60/20/20 into train/evaluation/test. Hyperparameter search and cross-validation used only the training portion with 5-fold CV (Kohavi, 1995). The evaluation split informed model selection; the test split remained untouched for final confirmation. We reported MAE, Median AE, RMSE, and R^2on Train and Eval for every model to diagnose over/under-fit.

To maximize training efficiency, CV ran with parallel folds (n_jobs=-1) while inner estimators were kept single-threaded processing during searches to avoid thread oversubscription. Where searches were lengthy, we adjusted trial budgets and, if necessary, escalated to a higher-resource environment.

Baseline non-linear model: Decision Tree with grid search

We established an interpretable baseline with a DecisionTreeRegressor, tuned via GridSearchCV over max_depth, min_samples_leaf, max_leaf_nodes, and ccp_alpha. The objective was twofold: (i) map primary breakpoints (e.g., distance thresholds) that align with operational intuition; and (ii) calibrate a reference for variance-reduced ensembles. Despite pruning, the tree predictably underfit residual structure but provided valuable threshold insight for later models.

We encoded calendar and other discrete signals with one-hot encoding (OHE) to capture non-monotonic effects while avoiding spurious ordinality (e.g., weekday $0 \nless 1 \nless \ldots \nless 6$). Concretely, we applied a ColumnTransformer with OneHotEncoder to purchase_weekday (7 levels) and purchase_month (12 levels), while treating purchase_is_weekend as a binary factor with one column dropped to prevent collinearity. This ensured fold-consistent design matrices (unknown levels at eval/test are safely ignored), enabled models that require purely numeric inputs (e.g., XGBoost), and preserved interaction flexibility (e.g., distance × specific weekday). For LightGBM, which natively handles categoricals, we instead passed categorical indices to avoid unnecessary dimensional expansion, keeping OHE only where it provided modeling or interoperability benefits.

Random Forest for variance reduction on tabular data

We then evaluated a RandomForestRegressor (Breiman, 2001) to reduce variance via bagging and random subspace sampling, which is well-suited to tabular, heterogenous signals such as distance × calendar × freight interactions. We tuned n_estimators, max_depth, min_samples_leaf, and max_features using 5-fold CV on the training partition of the 20% subsample. RF consistently improved Eval-MAE over the single tree and served as a strong baseline prior to boosting.

Gradient boosting at scale: XGBoost and LightGBM with Bayesian tuning

To capture structured residuals beyond bagging, we trained XGBoost (Chen & Guestrin, 2016) and LightGBM (Ke et al., 2017). Because exhaustive grids scale poorly in boosted spaces, we used Optuna's TPE (Akiba et al., 2019) to sample compact, high-return hyperparameter regions:

- XGBoost: eta (learning rate), n_estimators, max_depth, min_child_weight, subsample, colsample_bytree, gamma, reg_alpha, reg_lambda.
- LightGBM: learning_rate, num_leaves, min_data_in_leaf, feature_fraction, bagging_fraction/freq, lambda_l1, lambda_l2, min_gain_to_split.

Searches ran with 5-fold CV on the training split of the 20% subsample. We tuned trial budgets (n_trials≈40–80) to balance marginal gains and wall-clock, expanding only when an early lift suggested further potential. Both GBMs outperformed RF on Eval-MAE, with LightGBM exhibiting particularly favorable stability-to-accuracy trade-offs.

Stacked LightGBM + GAM with Ridge meta-learner

Given LightGBM's strong performance and the domain expectation of smooth, monotone-like effects (e.g., travel distance, order value, seasonal terms), we constructed a two-level stack:

1. Base learners: a tuned LightGBM and a GAM with spline terms for continuous features and a factor term for purchase_is_weekend (Hastie & Tibshirani, 1990; Wood, 2017).
2. Meta-learner: Ridge regression on out-of-fold (OOF) base predictions to blend learners while regularizing meta-weights and preventing overfit to the evaluation split.

GAM training considerations. During tuning, SCV convergence warnings arose because GAM penalization is sensitive to feature scales. We therefore standardized the GAM branch (stack class handles per-fold scalers) which stabilized convergence and improved out-of-fold error. A local resource crash during GAM optimization prompted a move to Google Colab for the Optuna search; tuned λvalues were then fixed and brought back into the stack. The final stacked model preserved LightGBM's accuracy while injecting interpretable smooth trends, improving Eval-MAE and reducing large-error frequency relative to single learners.

Managing model complexity and learning from iteration

Complexity controls were explicit and model-appropriate: pruning and ccp_alpha (tree), leaf size and depth (RF), structural and L1/L2 regularization with stochastic subsampling (GBMs), and shrinkage at the meta layer (Ridge). We discarded candidate configurations that improved Train scores without commensurate Eval gains or that worsened RMSE (tail risk).

Importantly, we complemented global metrics with slice performance (e.g., by purchase_weekday, purchase_month, and distance deciles) to verify that uplift generalized to operationally critical segments, not just the overall average. This rigorous, structured experimentation yielded a clear evolution: Single Tree → RF (variance reduction) → GBMs (residual capture) → Stacked LightGBM+GAM (accuracy + interpretability), culminating in a model family that met accuracy targets while remaining auditable for stakeholders.

Model Evaluation

This section aims to evaluate the performance and robustness of the trained models using MAE, RMSE, and R^2. It also examines model robustness and error distribution to assess generalization ability and discusses the corresponding business implications.

Evaluation Framework & Appropriate Metrics

The models were evaluated using a 20% test set, starting with 20% of the overall dataset instead of training on the full data.
When conducting model evaluation, it is essential to select appropriate metrics. In this study, MAE, RMSE, Median AE, and R^2 were adopted as the key evaluation measures. These metrics capture different dimensions of model performance and provide distinct business implications. As shown in the table below, MAE provides a direct measure of the average prediction error, which is easy to interpret in business contexts such as the typical deviation in predicted delivery time. It is particularly useful for assessing accuracy from a customer perspective.
Median AE takes the median of all absolute errors rather than the mean value, making it more robust to outliers. It reflects the stability and reliability of predictions under normal conditions; a lower Median AE indicates more consistent delivery time forecasts.
RMSE penalizes larger errors more heavily, making it more sensitive to outliers and better suited for identifying models that perform poorly under extreme conditions. This is particularly important for Olist's operations, where large delivery delays have significant impact.
R^2 measures the proportion of variance explained by the model, capturing its explanatory power and ability to represent the relationship between predictors and delivery time.
Together, these metrics offer a balanced and comprehensive evaluation of accuracy, robustness, and explanatory power, ensuring that the chosen model performs well both technically and in delivering actionable business insights.

Metric	Definition	Business Focus
MAE (Mean Absolute Error)	The mean of the absolute error between predicted and actual delivery times. It measures the distance from the predictions to the actual values on average.	MAE indicates the prediction error. Lower MAE means more accurate prediction on the average delivery time.
Median AE	Same residuals as MAE but takes the median.	Median AE is more robust, and it can reflect the typical errors of most samples.
RMSE (Root Mean Squared Error)	The square root of the mean of squared prediction errors. It penalizes larger errors more than smaller ones.	RMSE indicates the reliability of the prediction. Lower RMSE means fewer extreme errors.
R^2	Represents how much variance in actual delivery times can be explained by the model (from 0 to 1)	R^2 indicates the explanatory power of the model. Higher R^2 means the

		model captures more underlying patterns.

Evaluation Results

Comparison of models

Firstly, Optuna was employed to determine the optimal hyperparameters for each model. After identifying the best hyperparameters, five models with distinct configurations were trained and compared based on their evaluation metrics to identify the most effective one. A Python script was used to generate the table below, summarizing the evaluation results of all models.

```
=== TRAIN metrics (lower is better except R2) ===
                 Model      MAE  Median_AE     RMSE       R2
      XGBoost (Optuna) 4.123079   2.980680 6.448284 0.566961
 RandomForest (Optuna) 4.646315   3.295013 7.796380 0.366969
     Stacked (LGB+GAM) 4.802587   3.427517 7.983512 0.336215
     LightGBM (Optuna) 4.867103   3.470290 8.088609 0.318624
 DecisionTree (pruned) 5.159675   3.663907 8.496065 0.248248

=== EVAL metrics (lower is better except R2) ===
                 Model      MAE  Median_AE     RMSE       R2
     Stacked (LGB+GAM) 5.087543   3.582851 8.524038 0.248516
     LightGBM (Optuna) 5.118371   3.556144 8.549628 0.243997
 RandomForest (Optuna) 5.161782   3.606688 8.577411 0.239075
      XGBoost (Optuna) 5.223811   3.577182 8.704984 0.216272
 DecisionTree (pruned) 5.320678   3.746369 8.752104 0.207765

=== TEST metrics (lower is better except R2) ===
                 Model      MAE  Median_AE     RMSE       R2
               Stacked 5.128919   3.657179 8.052984 0.256466
       LightGBM (base) 5.155533   3.652410 8.083378 0.250843
```

From the evaluation results, the models show distinct strengths in terms of accuracy, robustness, and interpretability. Overall, the best model trained is the Stacked model (LightGBM+GAM), which performs best on the test set. It achieves the lowest MAE of 5.0857 and the lowest RMSE about 8.524, indicating that the smallest avreage and extreme prediction errors, and the highest R^2, showing that it can explain more variance in delivery time. Its Median AE (3.5829) is also almost identical to LightGBM's (3.5561), meaning the typical delivery prediction error is very close.

On the test set, the Stacked model achieves the lowest MAE (5.13) and RMSE (8.05), indicating both minimal average and extreme prediction errors. Its R^2 (0.256) is also the highest, suggesting it explains the greatest variance in delivery time. The Median AE (3.66) is close to that of the LightGBM base model (3.65), showing that both models produce similar accuracy for typical deliveries, but the stacked model performs better overall when accounting for outliers.

While LightGBM (Optuna) performs slightly better on typical cases, its overall MAE and RMSE are higher, suggesting that it is less stable under extreme delay conditions. Random Forest shows balanced but moderate performance, while XGBoost (Optuna) demonstrates clear overfitting — it achieves excellent training accuracy ($R^2 = 0.567$, RMSE = 6.45) but drops on the test set ($R^2 = 0.216$, RMSE = 8.70) significantly. Decision Tree serves as a weak baseline with the lowest accuracy across all metrics.

In summary, the Stacked model achieves the best balance between average error, typical error, and robustness to outliers. For Olist's delivery time prediction, this means more reliable and stable forecasts, reduced risk of severe delays, and stronger overall customer satisfaction, making it the most suitable model for real-world deployment.

Best model evaluation

The final stacked model was tested on test set and here are two additional evaluations: SHAP which shows how each feature pushes an individual prediction up or down from the model's baseline. On a stacked model, SHAP helps explain why the final ensemble predicts a certain delivery time. And Slice performance. The second is Slice performance, which computes MAE within business-related groups. This tells where the model is reliable and where may need buffers or fallbacks.（in Appendix A）

SHAP analysis identified *distance_log* and *purchase_month* as the two most influential predictors, followed by *freight_log* and *volume_log*. A longer delivery distance and certain purchase months (e.g., year-end periods) substantially increase predicted delivery time, suggesting strong effects from logistics and seasonality. Meanwhile, *purchase_weekday* and *payment_value_log* have moderate but consistent effects, reflecting operational differences between weekdays and weekends. The SHAP visualization provides interpretability by illustrating how high or low feature values influence model predictions.

Slice performance results show that prediction errors vary systematically across business segments. The model performs best on Monday (MAE = 4.78) and slightly worse on weekends (MAE ≈ 5.35), likely due to irregular courier operations. Across months, the lowest MAE occurs in mid-year (June–September) and the highest at the end of the year (November–December), confirming seasonal fluctuations in delivery efficiency. Distance-based analysis reveals a clear gradient, where longer distances correspond to higher errors (MAE rising from 3.24 to 7.56).

Overall, these results indicate that the stacked model not only achieves strong predictive performance but also show that distance, seasonality, and timing shape delivery accuracy.

Business implications

The evaluation results show that the Stacked model offers the most reliable and accurate delivery time predictions, which can directly boost Olist's operational efficiency. By reducing the average and extreme prediction errors, this model can make more precise delivery estimates, thus increasing customer satisfaction and trust. Besides, more accurate predictions of delays enable the logistics team to allocate resources more effectively and proactively manage high - risk orders, ultimately supporting faster deliveries and better service quality.

Business insights

Key business insights

Distance is the dominant driver of delivery time.

Orders with higher distance_log values significantly increase predicted delivery duration. This confirms that geographical reach and logistics routing are key factors in late deliveries. To improve performance, the company should consider regional warehousing, carrier diversification, or route optimization for long-distance orders.

Seasonal purchasing patterns affect delivery efficiency.

The feature purchase_month shows notable variation, suggesting that orders during certain months (likely high-demand periods such as holidays or year-end sales) lead to longer delivery times. Better capacity planning and temporary workforce scaling during these periods could reduce bottlenecks.

Freight and product volume correlate with longer deliveries.

Higher *freight_log* and *Volume_log* values are linked with extended delivery durations, likely due to larger or heavier shipments requiring special handling. Businesses can address this by segmenting shipping partners for bulky items or offering premium delivery options for heavy goods.

Timing of purchase also influences delivery performance.

The impact of *purchase_weekday* and *purchase_is_weekend* indicates that orders placed late in the week or on weekends tend to take longer to dispatch. Streamlining order processing schedules or enabling weekend warehouse operations could shorten fulfillment time.

Actionable recommendations

Optimize Logistics for Long-Distance Deliveries

- Establish **regional fulfillment centers** in high-demand areas to reduce shipping distances and lead times.
- Partner with **multiple carriers** or adopt **dynamic route optimization** to improve reliability for long-distance orders.
- Implement **predictive delay alerts** using the model's distance-based insights to inform customers proactively.

Strengthen Capacity Planning During Peak Seasons

- Use **historical purchase_month patterns** to forecast demand surges during holidays or sales events.
- Introduce **temporary workforce scaling** or **automated scheduling** to handle seasonal volume spikes efficiently.
- Promote early shopping for special occasion.

Differentiate Shipping Strategies by Product Type and Size

- Create **tiered logistics workflows**: fast-track small/lightweight products while assigning specialized handling for bulky or fragile items.
- Negotiate **custom service levels** with carriers for oversized shipments to minimize delay risk.
- Add a "buffer" time into the delivery estimation

Enhance Operational Flexibility in Order Processing

- Expand **warehouse operation hours** or enable **weekend processing** to mitigate delays caused by orders placed late in the week.
- Integrate order scheduling systems that align dispatch frequency with peak purchase days.

Limitations of insights and assumptions

There are several key limitations and caveats that should be noted. To begin with, the term *"stacked"* refers to the use of multiple models to make predictions, allowing the system to learn how to balance their complementary strengths. Specifically, the Generalized Additive Model (GAM) captures smooth and interpretable global trends, while LightGBM captures complex nonlinear interactions. However, training two models simultaneously significantly increases computational cost, leading to higher time and resource consumption.

From Olist's internal control perspective, particularly given its focus on serving small- to medium-income sellers while maintaining and updating the operational system), this elevated computational demand could place strain on the budget. Furthermore, high computational costs may also restrict the data scalability of the approach, as the existing system might not be capable of handling large datasets efficiently while simultaneously refining predictions through the stacked model

Another limitation is feature shift, which occurs when a new customer's feature patterns differ significantly from those in the training dataset. This issue is amplified because the current dataset lacks several important operational variables, such as postman delivery routes, the number and spatial distribution of delivery stations, freight cost, and transportation mode (e.g., road or sea), all of which strongly influence delivery time. Without these features, the model cannot fully capture the operational variability between customers, leading to less reliable predictions when applied to new or structurally different cases.

A key limitation is that the models were trained on a reduced dataset due to limited processing capacity and runtime. The team was unable to train on the entire dataset or conduct extensive hyperparameter tuning across all models. Using only part of the data may have restricted the model's exposure to the full variability of delivery patterns, potentially affecting generalization. Future work should consider using the complete dataset to achieve more robust and reliable performance.

Business implementation & Impact

Implement Plan

Process

First, as the prediction model is aimed at estimating the delivery time for the new customers and providing a more accurate shipping time estimate for better customer satisfaction, we can clarify the business goal before implementing the plan. Then, the model can be embedded within Olist's order management system. Once a new order is placed, the system will automatically update the output predictions. In terms of technology update, a weekly or monthly retraining schedule is proposed to make sure the model can keep up with new data, refreshing the model consistently.

Finally, it is not only a technological problem, but the most significant part is also about business value and impact. After operating the model, we must monitor the key performance factors weekly or monthly to track the model's performance and acquire some feedback from users and customers, ensuring the model has truly addressed the business issues and has brought more benefits to the company.

Organizational Integration

To accomplish the purpose of cooperation, all departments within the organization should use this approach. Additionally, the operation team may use this model to anticipate locations with greater delivery times by integrating it into the logistics planning dashboard. Drivers and warehouses can also benefit from this, as the average delivery delay time can be decreased with the right setup.

This model may also be used by the customer service department to forecast more accurate data, giving clients who have placed purchases in more specific delivery timeframes. Additionally, this has greatly decreased consumer complaints during the post-purchase phase.

Additionally, the production and marketing teams can divide the client base according to the anticipated delivery time in order to improve Olist's user retention skills. Although our model can only estimate the delivery time rather than the delay situation, we can nevertheless group

consumers into areas with shorter and more reliable expected delivery times, so demonstrating the dependability of Olist's services. Customers who anticipate lengthier delivery periods, on the other hand, can profit from compensating reductions, guaranteeing and preserving their contentment.

Business Impact

The increase in model accuracy may directly affect Olist and result in significant financial gains.
The assessment metrics of various models range greatly, as can be seen from the above table. With an R^2 value of 0.28 and an average absolute error (MAE) of around 5, the stacked LightGBM and GAM model is the one we have determined to be the best. This enhancement suggests that Olist can improve resource allocation, lower operational uncertainty, and more precisely forecast the delivery time.

Additionally, there are business benefits to the enhanced delivery time forecast. It may improve communication between suppliers, carriers, and customers and increase the dependability of logistics. Additionally, by using this strategy, Olist may minimise customer complaints by more efficiently allocating resources and making targeted deliveries during busy times. The order tracking function allows customers to obtain the exact delivery time of projection. Lastly, precise forecasts can boost customer loyalty and boost the company's standing.

Limitation and Risk

There are still certain risks and limits to the model. For example, the model may diverge owing to seasonal variations, seller behaviour, or unanticipated logistics events. Olist must employ fixed indications for testing and make sure the model is routinely maintained in order to guarantee ultimate dependability. Maintaining consistent data quality is essential for the model's accuracy because the data originates from a variety of sources.

All things considered, this application shows how the model helps the enterprise's overall operations. Delivery forecast time is integrated into Olist's everyday operations, which can improve customer confidence and avoid ineffective circumstances. This is a crucial component of Olist's digital transformation as it plays a crucial role in the relationship between data, prediction, and business.

Reference

Balan, G. S., Kumar, V. S., & Raj, S. A. (2025). Machine learning and artificial intelligence methods and applications for post-crisis supply chain resiliency and recovery. *Supply Chain Analytics*, *10*, 100121. https://doi.org/10.1016/j.sca.2025.100121

Kherbach, O., & Mocan, M. L. (2016). The Importance of Logistics and Supply Chain Management in the Enhancement of Romanian SMEs. *Procedia - Social and Behavioral Sciences*, *221*, 405–413. Sciencedirect. https://doi.org/10.1016/j.sbspro.2016.05.130

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623–2631.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Cover, T. M., & Thomas, J. A. (2006). *Elements of information theory* (2nd ed.). Wiley. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer.

Hastie, T., & Tibshirani, R. (1990). *Generalized additive models*. Chapman & Hall/CRC.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., … Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems, 30*.

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *IJCAI*, 1137–1145.

Wood, S. N. (2017). *Generalized additive models: An introduction with R* (2nd ed.). Chapman & Hall/CRC.

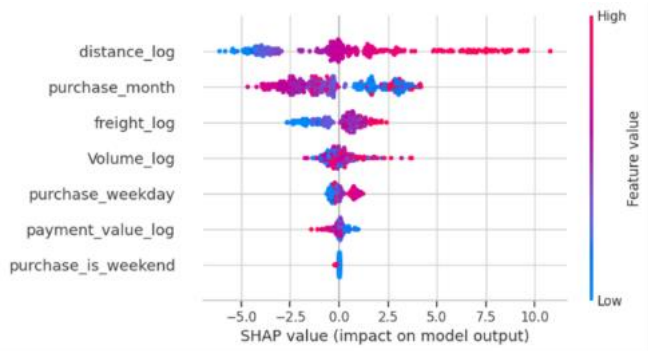
Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

Facey, O. (2023, September 14). *AI in Logistics and Last-Mile Delivery | Discover DHL*. Dhl.com. https://www.dhl.com/discover/en-global/logistics-advice/logistics-insights/ai-in-logistics-and-last-mile-delivery

Stiltner, M. (2024, March 14). *What is Boleto? here's everything you need to know*. Rapyd. https://www.rapyd.net/blog/what-is-boleto/

Appendix A: Model Evaluation

```
=== Top features by mean |SHAP| (Stacked, TEST sample) ===
            Feature  Mean|SHAP|
       distance_log    2.859605
     purchase_month    1.987220
        freight_log    0.972554
         Volume_log    0.482302
   purchase_weekday    0.410751
  payment_value_log    0.204916
purchase_is_weekend    0.036620
```



```
=== Slice MAE by purchase_weekday (TEST) ===
purchase_weekday
1    4.783590
0    5.093725
6    5.114671
3    5.167156
2    5.211879
4    5.298508
5    5.353347
dtype: float64
```

```
=== Slice MAE by purchase_month (TEST) ===
purchase_month
8     3.709926
7     3.808751
9     3.945501
6     4.299642
10    4.671653
4     4.750353
5     5.008119
2     6.286615
11    6.339109
1     6.471900
3     6.504678
12    6.755412
dtype: float64
```

```
=== Slice MAE by distance_decile (TEST) ===
(-6.909000000000001, 3.516]    3.238665
(3.516, 4.78]                  3.322653
(4.78, 5.649]                  4.074059
(6.1, 6.31]                    4.725790
(5.861, 6.1]                   5.121874
(6.31, 6.588]                  5.453143
(6.588, 6.803]                 5.727266
(5.649, 5.861]                 5.876396
(6.803, 7.294]                 6.188169
(7.294, 8.105]                 7.561177
dtype: float64
```

Appendix B: Teamwork & project execution

Due to schedule conflicts and personal preferences, most of our communication and collaboration took place online. To gather team members' perspectives on the Biggest Challenge**,** Most Impactful Strategy, and Key Takeaway for Future Career, we used Google Forms in the form of a survey and shared the link with all members. After collecting responses, we organized the findings to highlight areas of consensus while also summarizing differing viewpoints. This approach allowed us to distil collective insights and ensure that all voices were reflected in the final analysis. Here are the response we collected from Google Forms shown as images below:

Figure 1: Responses collected from members in terms of Individual Reflection with names

Form_Responses

Timestamp	How did you contribute to your team's success? Beyond task completion, how did your communication, problem-solving, or leadership improve the team's efficiency or cohesion? Provide an example of how your contribution helped your teammates or enhanced the project outcome.
11/7/2025 12:19:20	Jasmine(Hanxiao Hua):I did the model selection with hughson and run the GAM model, also the evaluation part. For the report, I did the evaluation and implementation&impact part.
11/7/2025 13:10:28	Malina Qin:I contributed not only by completing my assigned tasks but also by enhancing overall team efficiency and cohesion through proactive communication and problem-solving. For instance, during the feature engineering and data integration stage, I noticed that some team members were duplicating work on cleaning and aggregating tables, which risked inconsistencies. I suggested a structured workflow, dividing responsibilities clearly. I also created a consolidated guide for merging and aggregating features at the order level, which helped teammates understand dependencies between tables and reduced repeated efforts
11/7/2025 14:01:08	Mai Vu: I communicate clearly and reach out to my team whenever I have a question, have new insights, and ask if they need help. Before conducting the final report, I also ask them before editing their part.
11/7/2025 16:42:50	Hughson Xu: I have successfully completed my coding part to evaluate the performance of different models. The best model will forecast the delivery time accurately and provide insight to business of which part of the operation can be improved. During the entire project, I have actively communicating with other team members about my status, I also host frequent meeting to discuss the problems I encountered. The delivery forecast data I provided can be used by other team members to make suggestion to the company of what types of order should be prioritized.

Figure 2: Responses collected from members in terms of Group Reflection without names

Form_Responses

Timestamp	Biggest Challenge: Describe the biggest challe communication-based, or process-related? Ho with a similar challenge in the future, what wo	Most Impactful Strategy: What was the most i strategy or workflow improvement your team r how it improved efficiency, collaboration, or pr this approach in future team projects or a worl	Key Takeaway for Your Future Career: What is learned from this project that you will take into experience shaped the way you approach tean execution in a professional setting?
11/7/2025 13:59:35	I think the challenge that my team faces the most is the lack of effective project management, communication with one of the members at the beginning stage, and process-related. The more we involve with the process of exploring the data, the more variable and processes we created, which made us revserse back to the previous steps again, which is time-consuming	Having checkpoints such as the draft report makes thing easier by providing stepping stones and keeping people on track	Have a clear plan and smaller deadlines to achieve so everything is on right track instead of last minute
11/7/2025 14:13:11	The biggest challenge I think is technical and communication-based. Its time-wasting that the model selection and building part need eda and feture engineering also selection first. As each member is responsible for different part and we all have other tasks at the same time, it is hard to follow up this one quickly, in early stages, this may cause errors and inefficiencies. To overcome this challenge, we organized short meetings frequently and created a shared documentation file. This helped us follow up our progress and reduce redundant debugging.	The most impactful strategy was implementing a structured project workflow using shared documentation file. We divided the project into — data preparation, model selection&evaluation, and report writing — and assigned clear tasks for each. This approach improved both collaboration and quality. For example, when testing the stacked LightGBM + GAM model, each member could independently run the model while ensuring results were tracked systematically. In future teamwork, I would apply this strategy again to promote transparency, accountability, and reproducibility across projects.	The key takeaway I got was the importance of clear communication and iterative coordination in team-based projects. Even strong technical models rely on effective collaboration and mutual understanding. Through this project, I learned how to divide complex tasks, and merge diverse analytical insights into a final solution. In my future career, especially in data analytics and business report, I will continue applying structured communication, shared documentation, and cross-functional collaboration to deliver more efficient and data-driven results.
11/7/2025 14:24:01	Our team's biggest challenge was integrating multiple large and messy datasets from different sources into a consistent, order-level format for analysis. The data had many missing values, inconsistent formats, and required careful aggregation across tables like orders, customers, sellers, products, and payments. To navigate this, we divided the tasks based on expertise, held frequent check-ins to align on data cleaning standards, and documented our steps to ensure reproducibility. In the future, we would implement stricter initial data validation and establish clear aggregation rules at the start to reduce back-and-forth corrections, which would save time and improve efficiency	The most impactful strategy our team implemented was creating a clear modular workflow where each member was responsible for specific tables or features, while maintaining a shared data dictionary and version-controlled notebooks. For example, one member focused on cleaning and aggregating order_items, another on customer and seller geolocation, and another on payments and reviews. This division reduced redundant work, ensured consistency in feature engineering, and made it easier to merge all clean tables into a final order-level dataset. In future projects or workplace settings, I would apply this approach by clearly defining responsibilities upfront, maintaining documentation, and regularly syncing progress to enhance collaboration and overall project quality	The most valuable lesson I learned from this project is the importance of structured collaboration and clear communication in complex, data-driven projects. Coordinating multiple data sources, ensuring consistency in feature engineering, and integrating individual contributions into a cohesive final dataset highlighted how critical it is to document work, share updates, and clarify responsibilities. This experience has shaped the way I approach teamwork by emphasizing proactive communication, transparency, and systematic project execution—skills I will carry into any professional setting to improve efficiency, reduce errors, and ensure high-quality outcomes
11/7/2025 17:07:28	It is process related because the whole project process is not entirely linear, even people handling the earlier part needs results from the latter part. Therefore I would suggest a more systematic work and time allocation to each member in the future projects.	We started with a simple tree, than apply various modern techniques on it to improve the performance, instead of working on completely different models without fine tune. With small teams, I think it is the best idea to work and improve on the same thing.	Instead of only looking into the accuracy results from models, we should also look at which input has larger contribution and focus more on them as it could also be used to improve business.

Group Reflection

a. Biggest Challenge:

The biggest challenge our team faced was communication-based. Some members did not frequently check notifications or respond promptly, partly because the submission deadlines for this project overlapped with other course assignments. Additionally, since not all members were Data Analytics majors, it was difficult to maintain a consistent pace in report writing, coding, and scheduling.

This issue first appeared during the Team Charter stage. We were unable to gather all members before preparation began, and despite multiple attempts to contact one member via Ed Discussion, email, and group announcements, the Team Charter was eventually completed by only four members. Similar communication issues continued during the final report stage. Since each section of the report depended on the next, delays in one part affected the rest. Moreover, the model-building process required detailed EDA and feature engineering, which

demanded ongoing adjustments due to issues such as missing values and inconsistent data formats. The more variables and steps we created, the more time we needed to backtrack and revise previous work. Effective summaries were also necessary to guide teammates and ensure the report remained concise and well-organized.

To overcome this challenge, we appointed a team leader responsible for ensuring that messages were delivered to all members. The leader-maintained communication through phone calls, shared group chats, and short Zoom or WhatsApp meetings. Screen sharing was used to highlight and resolve coding or documentation issues in real time.

If faced with a similar challenge in the future, we would prefer to work together in person whenever possible. Face-to-face collaboration ensures all members are on the same page, reduces information loss, and allows mistakes in documents or code to be detected and corrected promptly.

b. Most Impactful Strategy

The most impactful project management strategy our team implemented was establishing structured checkpoints throughout both the Jupyter Notebook and the written report. These checkpoints acted as stepping stones that kept everyone on track and ensured alignment between code development, analysis, and documentation.

To make progress more transparent, we inserted subheadings and topic labels in both the shared notebook and report draft. Each checkpoint represented a stage where code outputs and corresponding written interpretations were reviewed together. This system worked effectively alongside clear version control, improving both efficiency and accuracy.

For example, after completing the data preprocessing step, we created a checkpoint in the notebook and another in the report. Team members then verified that the figures, tables, and explanations matched the latest results. When a data cleaning issue was later identified, we were able to revert to the previous notebook checkpoint and quickly update the related report section—without having to redo the entire analysis. This significantly reduced redundancy and minimized the risk of inconsistent documentation.

We also combined this approach with a clear division of responsibilities in reports and coding. For instance, one member handled the cleaning and aggregation of *order_items ,orders, order payment, product, product category, customer and seller geolocation data*, and another focused on exploring deeper analysis in terms of EDA through those variables using clear format where non null values and consistent data type and making visualizations that support our findings where can be found in Appendix C and main-report. Similarly, when testing our stacked LightGBM and GAM, Random Forest and Trees model**,** each member could independently run and validate results while maintaining synchronization through shared checkpoints. The Table 1 shown below reflects the distribution of responsibilities in reports.

Distribution of work

Table 1: Distribution of work in report

- Section 1: Problem formulation & business understanding (approx. 1-2 pages) **Lia**
- Section 2: Exploratory data analysis (EDA) (approx. 2-3 pages) **Mai and Malina**
- Section 3: Feature engineering (approx. 1-2 pages) **Malina and Mai**
- Section 4: Recommended models & justification (approx. 2 pages) **Hanxiao and Hughson**
- Section 5: Model evaluation & performance analysis (approx. 2 pages) **Hanxiao and Hughson**
- Section 6: Business insights (approx. 1-2 page) **Lia and Mai**
- Section 7: Business implementation & impact (approx. 2-3 pages) **Hanxiao**
- Appendix A: Model development & experimentation (approx. 3-5 pages) **Hanxiao and Hughson**
- Appendix B: Teamwork & project execution **Lia**
- Appendix C: Additional tables and figures (no page limit) Except Lia

In future team projects or workplace settings, we would apply this strategy by defining clear responsibilities early, maintaining structured documentation with checkpoints, and scheduling regular progress syncs. This approach not only enhances collaboration and efficiency but also helps ensure high-quality, well-aligned project deliverables.

c. Key Takeaway for Your Future Career

The most valuable lesson we learned from this project is the importance of structured collaboration and clear communication in complex, data-driven work. Having a clear plan with smaller, achievable deadlines helped keep the project on track and prevented last-minute pressure.

Throughout the project, we realized that even the strongest technical models depend on effective teamwork and mutual understanding. Maintaining consistency in feature engineering and merging individual contributions into a cohesive final dataset when coordinating multiple data sources can demonstrating how essential it is to document work carefully, share progress regularly, and clarify responsibilities early on.

This experience has reshaped how we approach teamwork and project execution. In the future career—especially in data analytics and business reporting—we will apply structured communication, shared documentation practices and cross-functional collaboration to deliver more efficient, transparent, and data-driven results.

Individual Reflection

Table 2: The information of Group Member

SID	Name	Abbreviation
530263829	Malina Qin	MQ
540154801	Lia Xu	LI
430024450	Hughson Xu	HU

540615324	Mai Vu	MV
540321469	Hanxiao Hua	HXH

MQ: I noticed that some team members were duplicating work on cleaning and aggregating tables, which risked inconsistencies. I suggested a structured workflow, dividing responsibilities clearly.

HXH: I did the model selection with Hughson and run the GAM model, also the evaluation part. For the report, I did the evaluation and implementation and impact part.

MV: I communicate clearly and reach out to my team whenever I have a question, have new insights, and ask if they need help. Before conducting the final report, I also ask them before editing their part.

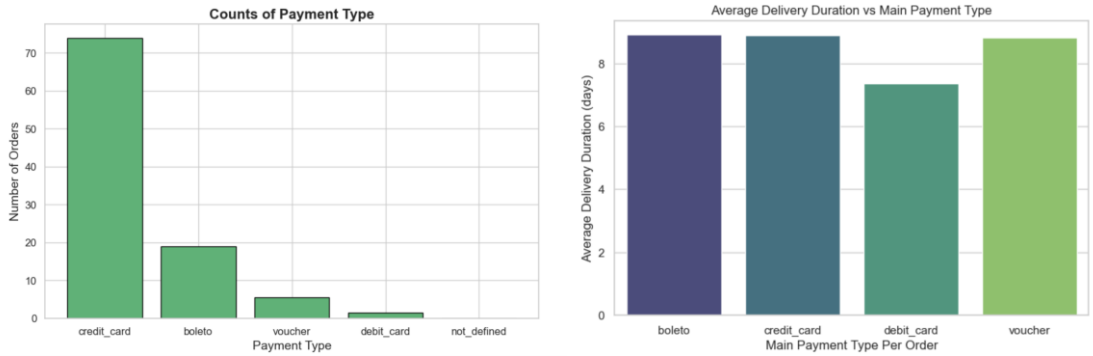
Lia: One of my main contributions to my team was improving our communication and coordination. I created a shared chat group and used the Ed discussion board to reach out to another teammate, ensuring that everyone stayed connected. I encouraged all members to regularly update their progress in the shared chat so that we could monitor our collective progress and support each other in time.

For collaborative tasks such as the *Group Reflection* and *Team Charter*, I took the initiative to organize meetings to consolidate everyone's ideas. I collected my teammates' availability preferences, finalized the meeting time, shared the Zoom link, and reminded everyone on the meeting day by tagging all members.

In terms of editing and task management, I set clear timelines and internal deadlines for each section to ensure that there was always material ready for review and that our progress stayed on track. These actions not only helped improve our workflow efficiency but also strengthened our team cohesion and accountability.

HU: I have successfully completed my coding part to evaluate the performance of different models. The best model will forecast the delivery time accurately and provide insight to business of which part of the operation can be improved. During the entire project, I have actively communicated with other team members about my status, I also host frequent meeting to discuss the problems I encountered. The delivery forecast data I provided can be used by other team members to make suggestion to the company of what types of order should be prioritized.

Appendix C: Additional tables and figures



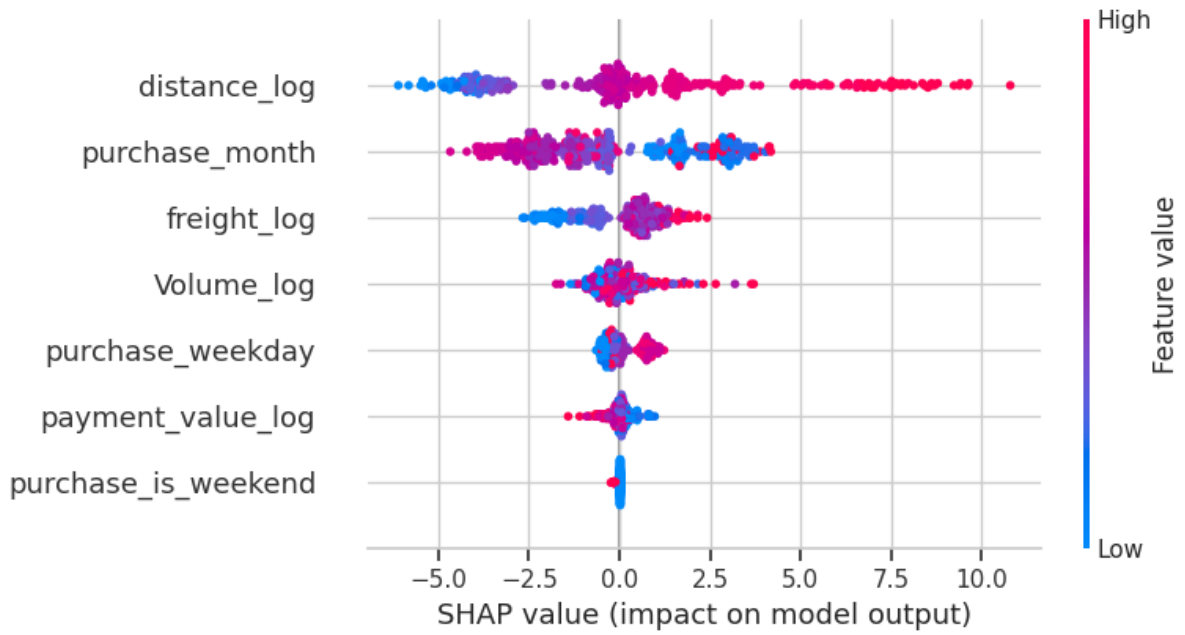
Appendix C. 1. Payment types count & average delivery time

```
=== TRAIN metrics (lower is better except R2) ===
                Model      MAE  Median_AE     RMSE       R2
     XGBoost (Optuna) 4.123079   2.980680 6.448284 0.566961
RandomForest (Optuna) 4.646315   3.295013 7.796380 0.366969
    Stacked (LGB+GAM) 4.802587   3.427517 7.983512 0.336215
    LightGBM (Optuna) 4.867103   3.470290 8.088609 0.318624
DecisionTree (pruned) 5.159675   3.663907 8.496065 0.248248

=== EVAL metrics (lower is better except R2) ===
                Model      MAE  Median_AE     RMSE       R2
    Stacked (LGB+GAM) 5.087543   3.582851 8.524038 0.248516
    LightGBM (Optuna) 5.118371   3.556144 8.549628 0.243997
RandomForest (Optuna) 5.161782   3.606688 8.577411 0.239075
     XGBoost (Optuna) 5.223811   3.577182 8.704984 0.216272
DecisionTree (pruned) 5.320678   3.746369 8.752104 0.207765

=== TEST metrics (lower is better except R2) ===
          Model      MAE  Median_AE     RMSE       R2
        Stacked 5.128919   3.657179 8.052984 0.256466
LightGBM (base) 5.155533   3.652410 8.083378 0.250843
     GAM (base) 5.269382   3.734968 8.560614 0.246375
```

```
=== Top features by mean |SHAP| (Stacked, TEST sample) ===
             Feature  Mean|SHAP|
        distance_log    2.859605
      purchase_month    1.987220
         freight_log    0.972554
          Volume_log    0.482302
    purchase_weekday    0.410751
   payment_value_log    0.204916
purchase_is_weekend    0.036620
```




```
=== Slice MAE by purchase_weekday (TEST) ===
purchase_weekday
1    4.783590
0    5.093725
6    5.114671
3    5.167156
2    5.211879
4    5.298508
5    5.353347
dtype: float64

=== Slice MAE by purchase_month (TEST) ===
purchase_month
8     3.709926
7     3.808751
9     3.945501
6     4.299642
```

```
10    4.671653
4     4.750353
5     5.008119
2     6.286615
11    6.339109
1     6.471900
3     6.504678
12    6.755412
dtype: float64

=== Slice MAE by distance_decile (TEST) ===
(-6.909000000000001, 3.516]    3.238665
(3.516, 4.78]                  3.322653
(4.78, 5.649]                  4.074059
(6.1, 6.31]                    4.725790
(5.861, 6.1]                   5.121874
(6.31, 6.588]                  5.453143
(6.588, 6.803]                 5.727266
(5.649, 5.861]                 5.876396
(6.803, 7.294]                 6.188169
(7.294, 8.105]                 7.561177
dtype: float64
```